

Projektportfolio

Willy Cai

+49 176 60987946

Ritterlandweg 54E, 13409 Berlin

22.07.1991

willy.cai@icloud.com

github.com/wiilee

TURNIERPLANER - DESKTOP APP

[GitHub Repository](#)

PROJEKTKONTEXT

Bisherige Turnierverwaltung erfolgte manuell mit Stift & Papier. Bei steigender Teilnehmendenzahl führte dies zu erhöhtem Administrationsaufwand und unruhigem Event-Ablauf, da eine visuelle Live-Anzeige fehlte, die den Teilnehmern den aktuellen Zustand des Turniers darstellt.

PROJEKTZIEL

Entwicklung und Implementierung einer plattformunabhängigen Desktop-Anwendung zur vollständigen Digitalisierung, Automatisierung und Live-Visualisierung des gesamten Turnierablaufs.

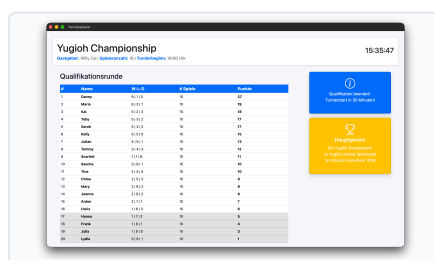
TECH-STACK

- Frontend:** HTML5, CSS3, Bootstrap, JavaScript, React
- Desktop-Framework:** ElectronJS (Node.js)

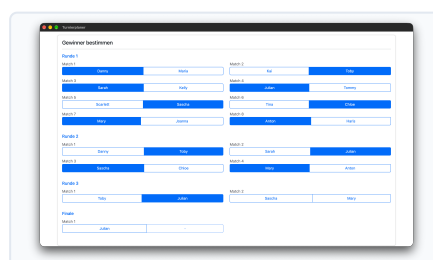
KERN-FEATURES & UMSETZUNG

- Zwei-Fenster-System:** Strikte Trennung von Adminview Fenster (Steuerung für den Veranstalter) und Mainview Fenster (maximierte Live-Anzeige für Beamer/zweiten Bildschirm).
- Dynamischer Hybrid-Turniermodus:** Flexibler Wechsel zwischen Qualifikationsrunde (sortierte Tabelle mit W/L/D-Statistik) und Turniermodus (klassische Turnierbäume für 8 oder 16 Personen).
- Echtzeit-Synchronisation:** Sofortige Übertragung von Spielständen, Tabellen-Updates und Paarungen von der Admin- auf die Mainview mittels IPC (Inter-Process Communication) in ElectronJS und React-State-Management.
- Personalisierung & Internationalisierung:** Unterstützung verschiedener Themes sowie mehrsprachige Bereitstellung (Deutsch/Englisch) mittels i18n.

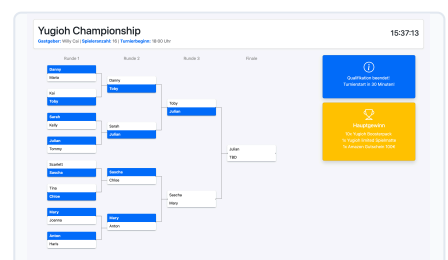
VORSCHAU & BENUTZEROBERFLÄCHE



Mainview - Qualifikation



Adminview - Steuerung



Mainview - Turnierbaum

PROJEKTKONTEXT

Im Rahmen des Projekts sollte eine KI-Wettkampfumgebung für die Strategiespiele „Racing Kings“ und „Jump Sturdy“ entwickelt werden, an deren Bereitstellung alle Teilnehmenden gemeinsam arbeiten sollen. Parallel dazu formierten sich die Teilnehmenden in 3er-Teams, um jeweils ein eigenes, autonom agierendes KI-Modell für eines der beiden Spiele zu implementieren.

PROJEKTZIEL

Das übergeordnete Ziel war die Kombination aus Infrastruktur- und Modellentwicklung: Einerseits die gemeinsame Entwicklung einer zentralen Webplattform zum Hosten, Visualisieren und Auswerten der Spiele, andererseits die Konstruktion einer eigenen, kampffähigen und wettbewerbsstarken KI, die sich in automatisierten Turnieren erfolgreich gegen die Modelle der anderen Teams behaupten kann.

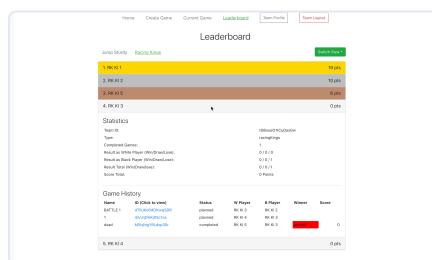
TECH-STACK

- **Frontend:** HTML5, CSS3, Bootstrap, JavaScript, React
- **Backend und Datenbank:** Python, Flask, C, MongoDB

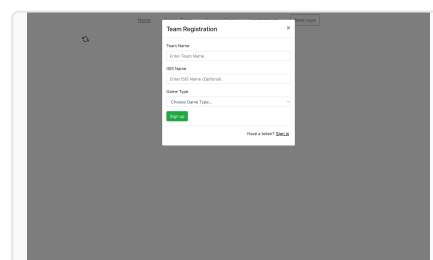
MEIN BEITRAG & UMSETZUNG

- **Authentifizierung:** Eigenständige Konzeption und Implementierung des Login-/Registration-Systems für die Plattform.
- **Visualisierung:** Mitwirkung bei der dynamischen Live-Darstellung des Racing-Kings-Spielfelds und der Spielfiguren.
- **Leaderboard:** Entwicklung und Umsetzung eines Ranking-Systems zur Performance-Auswertung und Ranglisten-Darstellung der konkurrierenden KI-Modelle.
- **KI-Entwicklung & Spieltheorie:** Konzeption und Implementierung eines autonomen KI-Modells in C innerhalb eines 3er-Teams. Fokus auf die Umsetzung des MiniMax-Algorithmus inklusive Alpha-Beta-Pruning zur effizienten Suchbaumreduzierung sowie Mitwirkung bei der Entwicklung der Heuristik.

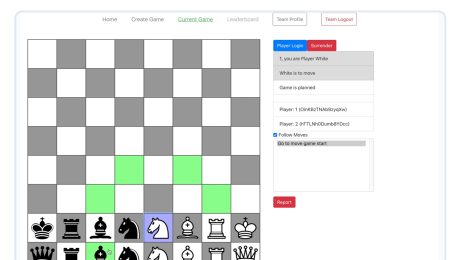
VORSCHAU & BENUTZEROBERFLÄCHE



Leaderboard



Login/Registrierung



Spielfeld für Racing Kings

PROJEKTKONTEXT

In vielen Arbeitsverhältnissen – insbesondere in Kleinbetrieben, der Gastronomie oder der Eventbranche – werden geleistete Arbeitszeiten vom Arbeitgeber oft fehlerhaft oder gar nicht erfasst. Beschäftigte greifen dabei häufig auf unstrukturierte Notizen zurück oder verzichten gänzlich auf eine eigene Dokumentation. Dies führt dazu, dass fehlerhafte oder fehlende Schichten nicht erkannt werden und bei Rückfragen keine verlässlichen Auskünfte gegeben werden können.

PROJEKTZIEL

Entwicklung einer plattformunabhängigen Web-App zur präzisen Schichtverwaltung für Arbeitnehmer. Die Anwendung erfasst Arbeitszeiten, Pausen, Trinkgelder sowie Zuschläge und berechnet automatisch Gesamtarbeitsstunden und Netto-Verdienst. Ein integrierter Schichtverlauf und PDF-Export ermöglichen den verlässlichen Nachweis geleisteter Schichten gegenüber dem Arbeitgeber.

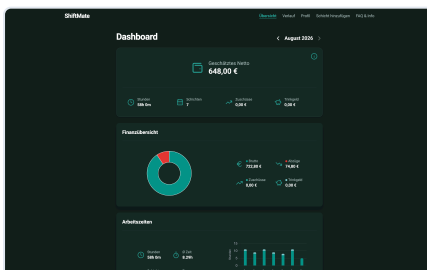
TECH-STACK

- **Frontend:** HTML5, CSS3, Tailwind CSS, TypeScript, Vue
- **Backend & Datenbank:** Node.js, Express, Supabase

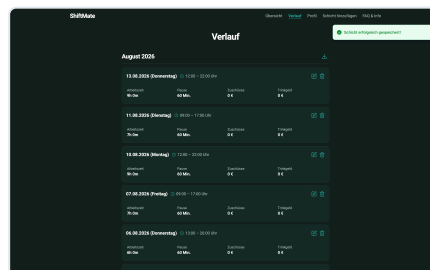
KERN-FEATURES & UMSETZUNG

- **Authentifizierung:** Implementierung eines Registrierungs- und Login-Systems mit Supabase Auth zur geschützten Speicherung persönlicher Nutzer- und Schichtdaten.
- **Präzise Zeiterfassung:** Eintragen von Arbeitszeiten mit Erfassung von Start, Ende und Pausenzeiten inklusive automatischer Berechnung der Gesamtarbeitsstunden mit Zuschüsse und Trinkgeld Funktion.
- **Netto-Gehaltsschätzung:** Integrierter Rechner zur dynamischen Ermittlung des voraussichtlichen Netto-Verdienstes basierend auf den tatsächlich geleisteten Schichten.
- **Schichtübersicht & Historie:** Strukturierte und übersichtliche Darstellung aller absolvierten Schichten zur Nachverfolgung der monatlichen Arbeit.
- **PDF-Export für Nachweise:** Funktion zum Generieren und Exportieren von detaillierten Schicht-Berichten als PDF-Dokument zur Klärung von Abrechnungsunstimmigkeiten mit dem Arbeitgeber.
- **Plattformunabhängigkeit:** Responsive Web-Anwendung zur mobilen und betriebssystemunabhängigen Nutzung direkt über den Browser ohne App-Installation.

VORSCHAU & BENUTZEROBERFLÄCHE



ShiftMate - Übersicht Seite



ShiftMate - Verlauf Seite

ShiftMate - Schicht einfügen

PROJEKTKONTEXT

Klassische Textverarbeitungsprogramme wie MS Word, Google Docs oder Apple Pages sind für die Gestaltung präziser A4-Seiten oft sperrig und erfordern mühsame Mausbedienung. Icons und Grafiken lassen sich meist nur eingeschränkt einbinden und Anpassungen von Farbe oder Formatierung sind oft nur über den Umweg importierter JPGs möglich.

PROJEKTZIEL

Ziel des Projekts ist die flexible Gestaltung von A4-Seiten auf Basis von HTML, CSS und JavaScript (Vue). Durch die volle JavaScript-Unterstützung lassen sich beispielsweise Daten-Arrays dynamisch per v-for-Schleife rendern. Dank der Anbindung etablierter Icon-Bibliotheken wie Lucide oder Material Icons können Symbole nahtlos eingebunden, skaliert und angepasst werden. Anstelle einer eigenständigen Anwendung wird ein Vite-Entwicklungsserver genutzt, um das Layout direkt in der bevorzugten IDE (z. B. VS Code) zu schreiben und Änderungen in Echtzeit im Browser einzusehen.

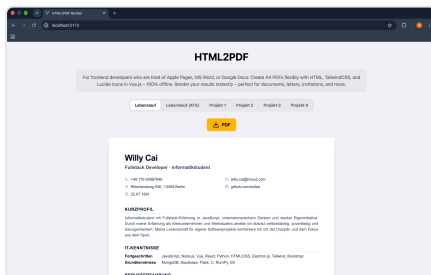
TECH-STACK

- **Frontend:** HTML5, CSS3, Tailwind CSS, TypeScript, Vue
- **Tooling & Build:** Vite, Node.js, VS Code

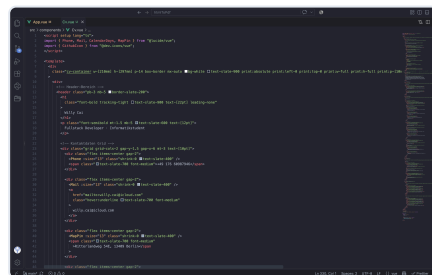
KERN-FEATURES & UMSETZUNG

- **Gestaltung mit HTML, CSS und JavaScript:** Volle visuelle Kontrolle über Layout, Typografie und Abstände ohne die Einschränkungen klassischer Textverarbeitungen. Erweiterbar mit beliebige JavaScript Bibliotheken.
- **Vorschau im A4-Format:** Direkte Live-Ansicht des Dokumentenlayouts im Browser mit exakter Skalierung auf die Proportionen einer echten A4-Seite.
- **Einfacher PDF-Export:** Schnelle Umwandlung des gerenderten HTML-Layouts in ein druckfertiges PDF-Dokument über einen einfachen Button.
- **Entwicklerzentrierter Workflow:** Nahtloses Zusammenspiel aus bevorzugter IDE (z. B. VS Code) für die Code-Eingabe und dem Browser als dynamischem Live-Monitor.

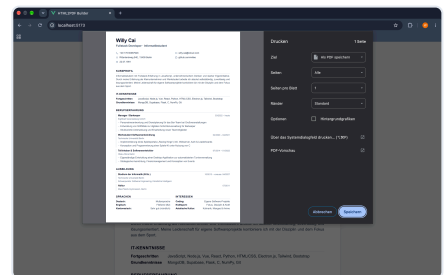
VORSCHAU & BENUTZEROBERFLÄCHE



HTML2PDF - Webansicht



VS Code - Code für A4 Seite



Export PDF Ansicht